

Clustering RNA sequences to identify cancer types

In this assignment, we will use clustering algorithms on RNA sequence data to identify cancer types. Since the [whole data](#) (from [Cancer Genome Atlas Pan-Cancer project](#)) is very big, we will use a [subset data from UCI Machine Learning repository](#). The subset data contains only 5 labels; BRCA, KIRC, COAD, LUAD and PRAD. The meanings of those labels are as below.

Abbreviation	Cancer
LUSC	Lung squamous cell carcinoma
READ	Rectum adenocarcinoma
GBM	Glioblastoma multiforme
BLCA	Bladder Urothelial Carcinoma
UCEC	Uterine Corpus Endometrioid Carcinoma
COAD	Colon adenocarcinoma
OV	Ovarian serous cystadenocarcinoma
LAML	Acute Myeloid Leukemia
HNSC	Head and Neck squamous cell carcinoma
LUAD	Lung adenocarcinoma
BRCA	Breast invasive carcinoma
KIRC	Kidney renal clear cell carcinoma

Although we can use the data for supervised learning model training, we will not use these labels for training, but use them for evaluation.

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import AgglomerativeClustering, KMeans
from sklearn.metrics import accuracy_score, confusion_matrix
import time
```

```
In [2]: # Read data. Do not change the variable names (data, label)
data = pd.read_csv('data/data.csv')
label = pd.read_csv('data/labels.csv')
data=data.drop('Unnamed: 0',axis=1)
label=label.drop('Unnamed: 0',axis=1)
```

A. Perform basic data inspection or EDA on the pandas dataframe.

- How many observations?
- How many features?
- If we were to train a "supervised" learning model, how would you deal with such large feature dimension?
- Even after feature dimension reduction, still the number of useful features may be enormous. How it would impact performance or runtime of certain supervised learning algorithms? Which algorithms would suffer from high dimension features than others and why?
- How it would impact performance or runtime of an unsupervised learning algorithm?
- Draw histograms of mean, max and min values in each feature. You may see numbers around 0-20. What those numbers mean? (We do not expect students to know or figure out the meanings, but if you do know by chance, feel free to discuss them with the class on the discussion board.)

```
In [4]: # YOUR CODE HERE

# number observations and features
print("Number of observations:", data.shape[0])
print("Number of features:", data.shape[1])

feature_means = data.mean(axis=0)
feature_max = data.max(axis=0)
feature_min = data.min(axis=0)

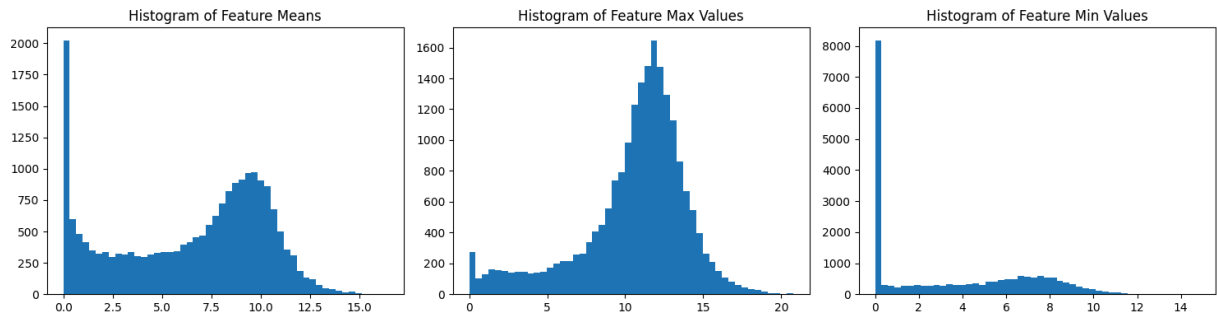
plt.figure(figsize=(15,4))
plt.subplot(1, 3, 1)
plt.hist(feature_means, bins=50)
plt.title('Histogram of Feature Means')

plt.subplot(1, 3, 2)
plt.hist(feature_max, bins=50)
plt.title('Histogram of Feature Max Values')

plt.subplot(1, 3, 3)
plt.hist(feature_min, bins=50)
plt.title('Histogram of Feature Min Values')

plt.tight_layout()
plt.show()
```

```
Number of observations: 801
Number of features: 20531
```



If we were to train a "supervised" learning model, we could handle so many features by applying dimensionality reduction. We could do that by using PCA to capture variance in fewer dimensions, feature selection to remove features with low variance or redundant ones, or lasso regression or tree-based models for embedded feature selection.

Impact on performance/runtime: High-dimensional data often leads to overfitting, increased training time, and memory inefficiency. Sensitive algorithms can perform poorly due to distance becoming less informative (KNN), can get very slow and overfit (SVM), and runtime can increase (decision trees/random forests).

Impact on unsupervised learning:

- distance metrics become less meaningful in high-dimensional space
- clustering becomes noisy and unreliable unless dimensionality is reduced first
- algorithms like k-means or hierarchical clustering may perform poorly without preprocessing

B. Build a hierarchical clustering model

Let's build a model using hierarchical clustering. Hierarchical clustering module is available from `sklearn.cluster.AgglomerativeClustering`. You can choose linkage type and metric. Please check its documentation for more details.

a) Number of clusters vs distance threshold

Oftentimes hierarchical clustering does not need to know the number of clusters in advance. Instead, one needs to choose threshold distance/similarity to cut the dendrogram later. The `AgglomerativeClustering` module lets you specify either the number of clusters (`n_clusters`) or the threshold (`distance_threshold`). Based on our data, which should we choose to set to which value and why?

- We should use `n_clusters=5` because the data is known to contain 5 cancer types and we can evaluate accuracy using known labels.

b) Guess which metric?

Can you guess which metric to use (distance-based vs. similarity-based) and why?
Share your thoughts with the class.

- We should use distance-based. Euclidean distance is the most common used distance measure for numerical, continuous gene expression data

c) Build a model

Build a model using `n_clusters=5` option. Choose any metric and linkage type at first. Display the clustering result labels (you can just print out the result). Do not change the variable (model) name.

```
In [5]: model=None
# YOUR CODE HERE
model = AgglomerativeClustering(n_clusters=5, linkage='ward', metric='euclidean')
model.fit(data)
print("Cluster labels:", model.labels_)
```

```
Cluster labels: [2 3 2 2 0 2 1 2 0 2 0 1 2 0 0 0 3 1 1 2 0 1 3 0 1 3 4 0 0 0
0 0 1 0 2 0 1
3 0 0 1 2 2 1 1 0 2 4 0 3 0 3 0 2 4 0 0 4 1 0 3 1 0 3 2 4 0 2 1 0 1 0 0 3
0 3 0 1 2 4 0 2 0 0 2 2 0 0 1 0 2 2 0 0 0 2 4 0 2 0 0 1 0 1 3 1 3 4 3 3 2
0 3 2 0 1 1 1 0 0 3 1 3 0 2 2 2 0 1 0 4 0 4 0 0 1 3 0 1 4 0 2 0 1 3 4 2 0
3 3 3 3 0 0 3 0 0 2 2 3 2 3 1 0 2 3 4 1 3 0 1 3 0 3 0 0 0 2 0 1 4 1 0 2 2
2 3 3 0 3 3 1 3 2 3 0 0 0 3 3 0 1 1 1 1 2 0 2 0 3 3 0 2 0 2 0 0 0 3 0 1 3
1 1 3 0 1 2 0 3 3 2 4 0 1 2 1 4 0 1 1 3 2 2 3 3 1 0 0 4 0 2 4 0 2 1 2 2 2
0 4 4 3 4 4 2 3 0 0 1 1 0 4 2 1 2 0 0 1 0 0 0 0 3 3 0 0 0 1 1 1 1 0 0 0 1
0 0 3 2 0 0 4 3 2 0 0 0 4 0 2 0 4 3 3 2 1 0 1 1 3 4 1 0 0 0 0 1 0 0 2 0 1
0 3 2 1 0 2 4 0 0 0 3 3 3 0 0 2 3 0 1 0 4 4 3 0 1 0 0 0 4 3 4 1 2 1 0 0 1
0 4 2 3 2 0 1 2 0 4 1 1 4 4 2 0 0 4 1 3 2 0 0 0 3 3 1 3 0 1 4 2 0 3 2 0 0
0 3 0 0 2 0 2 4 0 3 0 0 3 0 0 0 1 3 2 0 2 1 0 1 4 0 2 3 1 0 0 1 0 3 0 0 2
4 0 1 3 2 0 2 0 0 0 0 1 3 0 1 0 0 3 3 1 4 2 4 0 1 1 0 2 1 4 3 3 0 2 2 0 2
3 1 2 0 3 2 3 0 0 4 3 1 4 3 0 2 0 0 2 0 4 0 4 1 0 0 3 3 3 4 1 3 3 0 0 1 2
3 2 0 1 0 1 1 2 2 3 0 1 4 4 0 1 1 0 0 2 1 4 0 0 4 3 0 0 0 1 2 3 3 0 1 4 1
1 0 2 3 1 0 4 3 3 3 2 3 1 0 0 4 2 0 0 0 1 3 3 0 2 3 3 0 1 2 4 3 2 4 3 4 1
1 0 0 1 1 4 0 0 2 2 1 0 3 0 0 4 0 2 2 0 0 4 0 1 0 0 4 0 2 0 0 1 2 3 0 0 1
0 0 0 0 0 4 3 3 0 0 0 2 0 0 1 3 3 1 1 3 1 4 0 4 1 0 0 2 2 2 3 2 2 4 0 0 4
3 1 0 1 4 0 0 0 2 3 1 0 2 1 2 0 3 1 2 3 2 2 0 1 2 3 4 4 0 0 0 3 1 1 1 0 3
1 2 0 3 2 0 2 0 1 0 4 2 2 1 2 1 0 3 3 0 0 1 0 0 0 0 1 1 2 4 1 0 0 1 0 3 0
0 2 0 2 0 4 0 0 1 3 0 0 2 0 2 4 0 0 0 3 0 3 0 3 1 1 4 4 0 0 0 3 0 3 1 0 3
1 3 3 3 1 0 2 0 0 1 0 2 1 0 0 0 0 2 3 0 3 3 2 2]
```

d) Label permutation

In clustering, the labels get assigned randomly, so the label numbering won't match the ground truth necessarily. Write a function below to find best matching label ordering based on the accuracy. Do not change the variable names.

Note: Label order is ['PRAD', 'LUAD', 'BRCA', 'KIRC', 'COAD'] as 0,1,2,3,4. So PRAD is 0, LUAD is 1, etc. This part asks you to rematch the labels from your model. The clustering

model does not know which label is which, but will give you 5 numeric labels (0-4). You'll need to find out which label correspond to which of the data set label.

```
In [9]: import itertools

def label_permute_compare(ytdf, yp, n=5):
    """
    ytdf: labels dataframe object
    yp: clustering label prediction output
    Returns permuted label order and accuracy.
    Example output: (3, 4, 1, 2, 0), 0.74
    """
    # YOUR CODE HERE
    ytrue = ytdf.iloc[:,0].astype('category').cat.codes.values
    perms = list(itertools.permutations(range(n)))
    best_acc = 0
    best_perm = None
    for p in perms:
        yp_new = [p[i] for i in yp]
        acc = accuracy_score(ytrue, yp_new)
        if acc > best_acc:
            best_acc = acc
            best_perm = p
    return best_perm, best_acc
```

```
In [10]: labelorder, acc = label_permute_compare(label, model.labels_)
print(labelorder, acc)
```

```
(0, 2, 4, 3, 1) 0.9950062421972534
```

e) Check confusion matrix

Use sklearn's confusion matrix and display the results.

```
In [12]: # YOUR CODE HERE
yp_new = [labelorder[i] for i in model.labels_]
ytrue = label.iloc[:,0].astype('category').cat.codes.values
conf_matrix = confusion_matrix(ytrue, yp_new)

print(f"Confusion Matrix:\n {conf_matrix}")
```

```
Confusion Matrix:
[[299  0  0  1  0]
 [ 0 77  0  1  0]
 [ 0  0 146  0  0]
 [ 2  0  0 139  0]
 [ 0  0  0  0 136]]
```

f) Change linkage method and distance metric. Which ones lead the best performance? Print out the accuracy and confusion matrix for the best model.

```
In [15]: # YOUR CODE HERE
linkages = ['average', 'complete', 'single']
```

```

metrics = ['euclidean', 'manhattan']

best_acc = 0
best_config = None

for link in linkages:
    for metric in metrics:
        try:
            model = AgglomerativeClustering(n_clusters=5, linkage=link, metric=metric)
            model.fit(data)
            labelorder, acc = label_permute_compare(label, model.labels_)
            if acc > best_acc:
                best_acc = acc
                best_config = (link, metric, labelorder)
                best_model = model
        except Exception as e:
            print(f"Failed for linkage={link}, metric={metric}: {e}")

print("Best Hierarchical Model:")
print(f"Linkage: {best_config[0]}, Metric: {best_config[1]}")
print(f"Best accuracy: {best_acc}")
print(f"Label order: {best_config[2]}")

```

Best Hierarchical Model:
 Linkage: complete, Metric: euclidean
 Best accuracy: 0.9313358302122348
 Label order: (2, 0, 1, 3, 4)

C. What about k-means clustering?

Can we apply kmeans clustering on this data? Which clustering methods give a better performance? Is kmeans faster or slower?

```

In [16]: # YOUR CODE HERE
start = time.time()
kmeans = KMeans(n_clusters=5, n_init=10, random_state=42)
kmeans.fit(data)
end = time.time()

labelorder_kmeans, acc_kmeans = label_permute_compare(label, kmeans.labels_)
print(f"KMeans accuracy: {acc_kmeans}")
print(f"Runtime: {end - start}")

```

KMeans accuracy: 0.9937578027465668
 Runtime: 2.4725916385650635

In []: